Feature Track Summary Visualization for Sequential Multi-View Reconstruction

Shawn Recker*, Mauricio Hess-Flores*, and Kenneth I. Joy*
*Institute of Data Analysis and Visualization
University of California Davis, Davis, California 95616
Email: see http://www.idav.ucdavis.edu/people

***Abstract*—Analyzing sources and causes of error in multi-view scene reconstruction is difficult. In the absence of any ground-truth information, *reprojection error* is the only valid metric to assess error. Unfortunately, inspecting reprojection error values does not allow computer vision researchers to attribute a cause to the error. A visualization technique to analyze errors in sequential multi-view reconstruction is presented. By computing feature track summaries, researchers can easily observe the progression of feature tracks through a set of frames over time. These summaries easily isolate poor feature tracks and allow the observer to infer the cause of a delinquent track. This visualization technique allows computer vision researchers to analyze errors in ways previously unachieved. It allows for a visual performance analysis and comparison between feature trackers, a previously unachieved result in the computer vision literature. This framework also provides the foundation to a number of novel error detection and correction algorithms.**

I. Introduction

Multi-view scene reconstruction refers to the reconstruction of three-dimensional objects from a collection of images taken from known camera viewpoints. From each camera center, rays are projected through each corresponding image plane, which are intersected to find the three-dimensional (3D) point for corresponding pixels between the images. In order to accurately reconstruct a scene many sequential operations are performed. First camera calibration is performed to determine the cameras' intrinsic parameters, for example, focal length [1]. Then mappings between corresponding pixels, in different images of the scene are calculated. This feature mapping (two views) or feature tracking (across multiple views) can be sparse or dense [2], [3], [4]. The epipolar geometry can then be estimated from the given matches [1]. Pose estimation is performed to calculate the cameras' extrinsic parameters of translation and rotation [1]. Once the camera parameters are determined, linear triangulation is used to determine the 3D structure [1]. Often, errors in the previously mentioned steps influence the accuracy of the results. In such cases, bundle adjustment is performed to optimize the different parameters of this process [5].

Accurate feature tracks fundamentally determine the accuracy of a multi-view reconstruction, as this affects the subsequent camera intrinsic and extrinsic calibrations, as well as the computation of scene structure. Even when using robust estimation procedures and outlier detection, such as RANSAC [1], lighting conditions, occlusions, and repetitive patterns limit feature tracking efficacy and skew subsequent calibration and structure estimation. These stages are prone to additional sources of error and numerical instability. Furthermore, the absence of ground-truth camera and structure parameters forces multi-view algorithms to resort to non-linear optimization of parameters to reduce reprojection error in order to obtain accurate point clouds. However, high numbers of scene points and cameras can make such *bundle adjustment* an expensive element in a reconstruction pipeline, despite efficient sparse implementations [6].

Analyzing error in multi-view reconstruction is difficult. In the absence of ground truth data, *reprojection error* is the only valid metric to assess a reconstruction. Reprojection error is the sum of squares of the Euclidean distance between a computed structure position and its corresponding feature track locations across all frames in which the point is visible [1]. Computer vision researchers have difficulty analyzing reprojection error to determine the cause of such error. Traditionally, total reprojection error for a given scene reconstruction has been treated as a numerical quantity, where the goal has always been to minimize its value. However, total reprojection error does not provide any information regarding the sources of the error. For instance, a small number of feature tracks which present high reprojection error could be the main component of total reprojection error, and bundle adjustment optimization may not be able to lower its value much, even through its expensive computation. Minimization of reprojection error requires non-linear optimization of the sum-of-squares reprojection error function, which is prone to miring in local minima and may not converge to the global optimal solution. However, if inaccurate feature tracks could be removed or corrected, total reprojection error may be much lower, and bundle adjustment would converge much faster with a greater probability of obtaining the global minimum.

In light of this, this paper presents a feature track summary visualization framework that provides insight into the behavior of feature tracks used in a sequential reconstruction. Users are able to identify sources of error and easily observe poor feature tracks using the system in contrast to other mechanisms. The method is designed for structured or sequential sets of images, such that flow over time can be observed. For unstructured sets of images, it would be very difficult to visually observe trends over time. Results are shown and analyzed to demonstrate the application of this framework to common sequential scene reconstruction problems. By

encoding values, such as reprojection error, along individual tracklines, it is possible to perform a visual before-and-after analysis for bundle adjustment, where it becomes clear to see which tracks contribute most towards final total reprojection error. The effect of radial distortion on feature tracking and scene reconstruction is also analyzed. Finally, the performance of different feature trackers becomes possible, by analyzing the combination of track density, general flow of the tracks, and encoded error values. Related work will be detailed in Section II, followed by the algorithm in Section III, with results in Section IV, conclusions in Section V, and future work in Section VI.

II. Related Work

In scene reconstruction pipelines, feature tracking is a necessary step and perhaps the most important. Lowe's method using the SIFT feature descriptor [2] permits reliable feature matches across views. SIFT is scale and rotation-invariant, robust against noise, and robust against changes in illumination. Lowe's technique uses a database of 128 dimensional feature vectors to perform feature matching . The SURF algorithm, inspired by SIFT, in contrast, uses only a 64 dimensional feature vector for descriptor purposes [3]. The lower dimensionality of the feature vector permits faster matching at the cost of less reliability, however this technique is still scale and rotation invariant . Once the feature tracks are obtained, recovery of the extrinsic camera parameters is accomplished by estimating the epipolar geometry and applying Singular Value Decomposition on the essential matrix to compute the values [1]. Intrinsic camera parameters such as focal length are typically assumed known. Additional methods to calculate camera pose include the Direct Linear Transformation and using vanishing points and lines [1]. These methods frequently produce inaccurate data and require an additional non-linear optimization to attain higher quality results. This optimization, known as bundle adjustment, occurs when all structures and poses are available [5]. The Levenberg-Marquardt algorithm for bundle adjustment minimizes the reprojection error; however, the parameter space for the algorithm may not converge to the global optimum [6].

There are numerous algorithms in the computer vision literature based on the described pipeline stages. For example, Akbarzadeh et al. [7] introduced a method for dense reconstruction of urban areas from a video stream. Pollefeys et al. [8] used a similar approach for real-time urban reconstruction. Goesele et al. [9] presented a reconstruction pipeline for large, unstructured collections of online photographs of a scene, based on an adaptive view selection technique that robustly computes depth maps along with 3D models of the scene.

There has been some work on uncertainty analysis specifically for scene structure computation [1], [10], [11], [12], but it has been mainly a mathematical analysis which has not been enhanced by visualization techniques. For example, Rodehorst et al. [13] introduced a ground-truth based approach to evaluate camera pose estimation, while Knoblauch et al. [14] introduced a geometric error extraction of both feature matches and camera pose errors. This method does not rely on ground-truth data or any other assumptions about the scene.

Visualization of uncertainty has been applied to image segmentation and recently, scene structure computation. Torsney-Weir et al. [15] introduced a system which utilized uncertainty metrics to guide parameter values in image segmentation. Saad et al. [16] utilized a similar approach to medical image segmentation based upon probabilities. Recker et. al [17] introduced a system based upon an angular error metric to generate a scalar field that allowed for a visually enhanced numerical exploration of scene structure computation.

Many of the error analysis techniques in multi-view reconstruction focus on fixing errors present in a reconstruction. Knoblauch et. al present techniques to analyze the sources of errors, however, only for steroscopic reconstruction, not multi-view reconstruction in general. Recker et. al's method, while permitting analysis of multiple view reconstruction, only visualizes the uncertainty associated with a single structure point. This framework differs in that an entire reconstruction can be analyzed at one time, not just between two different frames or for a single feature track. The summary visualization technique relies on computing feature track lines (or "tracklines") between the set of frames and color mapping essential reconstruction information on each trackline. The ability to summarize large amounts of information very concisely provides a unique ability to analyze multi-view reconstruction parameter errors, and conceptualize techniques to fix errors.

III. Procedure

To generate the feature track summary visualization for a reconstruction, tracklines are computed to demonstrate the progression of a particular feature track throughout a set of images. The output is displayed in a visualization tool designed for the proposed analysis. Camera projection matrices, P; feature tracks, F; and computed structure positions, S, are provided as input to the visualization tool. The application computes a trackline corresponding to each feature track. A diagram depicting the construction of a trackline is depicted in Figure 1.

Mathematically, define $I = \{I_1, I_2, ..., I_n\}$ to be the set of all images. I_i is the set of keypoints in image i, so $I_i = \{\emptyset, k_{i1}, k_{i2}, ..., k_{im_i}\}$, where k_{ij} is a two-dimensional (2D) keypoint location in image i of the form $(x_{k_{ij}}, y_{k_{ij}})$ and $\emptyset$ a null element. $x_{k_{ij}}$ denotes the horizontal pixel location of keypoint k_{ij} and $y_{k_{ij}}$ denotes the vertical pixel location of k_{ij}. A trackline, $T_t \in T$ is defined as a set of size n (the total number of images in the reconstruction), where $T_t = \{k_{1t}, k_{2t}, ..., k_{nt}\}$. Now each feature track, $f \in F$, consists of a set of keypoints which spans I. The system computes a trackline, T_t for each f such that the set of keypoints between the two sets are the same.

Visualization of the tracklines begins by conceptually arranging the frames along the Z-plane with a user-specified distance apart. The system connects the polyline formed by the keypoints k_{it} for trackline T_t. In the event the keypoint is

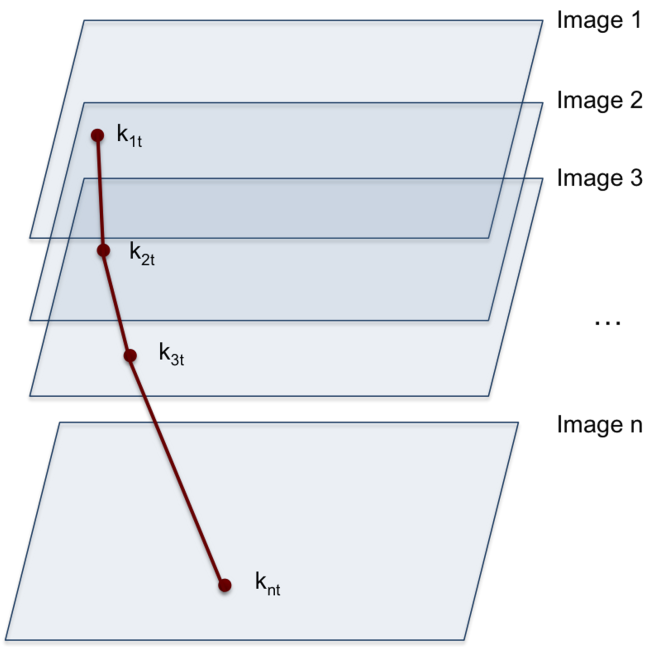


Fig. 1. This figure conceptualizes the creation of tracklines. A trackline, $T_t \in T$ is defined as $T_t = \{k_{1t}, k_{2t}, ..., k_{nt}\}$, a specific keypoint in each image. Now each feature track, $f \in F$, consists of a set of keypoints which spans I. The system computes a trackline, T_t for each f such that the set of keypoints between the two sets are the same.

the null element, no line is present in the visualization since the feature is not visible in the image. The line bundle is displayed in an interactive scene that can then be examined by the user.

The definition of a trackline can be expanded to include useful reconstruction information. Stated mathematically, the definition of $T_t = \{k_{1t}, k_{2t}, ..., k_{nt}\}$ becomes $T_t = \{\alpha_{1t}, \alpha_{2t}, ...\alpha_{nt}\}$, where α_{it} is a tuple containing the keypoint k_{it} and some number of scalar values. In this system, $\alpha_{it} = (k_{it}, \epsilon_r, \epsilon_a, \epsilon_b)$, where ϵ_r is the reprojection error associated with projecting $s_t \in S$ into image i using projection matrix $p_i \in P$ and measuring the distance with the projected point to keypoint k_{it}. The value ϵ_a is the angular error metric introduced by Recker et. al [17], and ϵ_b is the error metric introduced by Beder and Steffen [18], which is based on the roundness of the confidence ellipsoid for a computed 3D position. For Beder and Steffen's method, ϵ_b uses the information for image i and image $i+1$ since their metric requires image pairs. The various reconstruction information can then be visualized using a custom color map that maps ϵ values to colors.

1) Perceptual enhancements: In a large bundle of tracklines, individual tracklines are difficult to see or isolate. To address this issue, the application provides a mechanism to segment and view individual tracklines or some range of tracklines, in addition to the entire collection. An illumination model was implemented to enhance the visual perception of the trackline bundle. The program employs a Phong illumination model to increase photo-realism in our resulting



Fig. 2. This figure shows the resulting tracklines for the *Dinosaur* dataset [20] with the reprojection error color map. The visualization reveals areas of high reprojection error (shown in red). The high error feature tracks do not generally conform to the trackline flow of their nearby neighbors.

images. The illumination model is based upon Mallo et. al's formulation for illumination of streamlines [19].

IV. Analysis

The proposed tool permits the analysis of multi-view scene reconstructions and provides useful information to the user of the system. First, the influence of reprojection error on tracks in sparse and dense scene reconstruction are demonstrated for specific datasets. Then, comparisons between bundle-adjusted reconstructions and non-adjusted ones are discussed. Finally, an analysis and comparison of several successful and recent feature tracking algorithms was performed. All tests were conducted on a MacBook Pro machine with an *Intel Core i7* processor at 2.66 GHz with 4 GB of RAM, running Mac OS X Lion 10.7.3.

A. Tracklines for Sparse Scene Reconstruction

The Oxford Visual Geometry Group's well-known *Dinosaur* dataset [20] was examined. This dataset is commonly used as ground-truth comparison for new reconstruction algorithms. It contains 36 images of a turntable sequence of a toy dinosaur. The dataset contains ground truth feature tracks. After computing the tracklines for the dataset and examining the results, our visualization tool reveals interesting information. As indicated in Figure 2, there exist tracklines with high reprojection error (shown in red) which do not conform to the behavior of their nearby neighboring lines that have low reprojection error (shown in blue). The visualization of tracklines permits the user of the system to easily identify bad feature tracks.

B. Tracklines for Dense Scene Reconstruction

The visual analysis of feature track summaries included a dense reconstruction of the *Megascene1* synthetic dataset. Figure 3 shows the trackline visualizations along with one image used in the reconstruction. Duchaineau et al's dense correspondence algorithm [21] was used to obtain dense feature tracks for the reconstruction. This algorithm attempts

to match each and every pixel in a source image with one in a destination image, despite the fact that certain features may be occluded or no longer present in the next frame. This highly noticeable near the edges of the figure. Reprojection error is also higher in areas of occlusion (shown in green in part Figure 3(a)) due to the properties of the dense correspondence algorithm. This attribute is noticeable around the edges of buildings in the image.

C. Bundle Adjustment Comparison

A comparison between a reconstruction using bundle adjustment [5] with the SBA library [6] and one without bundle adjustment was performed. The reconstruction was generated using SIFT feature tracks [2] for the *Dinosaur* dataset. The resulting trackline visualizations are shown in Figure 4. It is easily observed that bundle adjustment improves the overal reprojection error for the reconstruction. For this reconstruction, there are a few tracks outside the main body of the dinosaur that are instantly observed to be incorrect. This is because the SIFT algorithm detected and tried to unsuccessfully match features that lie in the texture-less background of the *Dinosaur* images. Feature tracking with SIFT and basically all other algorithms tends to fail in such regions, due to the ambiguity in matching. This fact is noticeable since these tracks do not follow the general behavior of the others. The visual results in this comparison easily show the improvement to reconstruction that bundle adjustment obtains. However, it also shows that the final total reprojection error is non-zero due mainly to said tracks in texture-less regions. This method can reveal the cause of error, whereas reprojection error as a single scalar value cannot provide.

D. Radial Distortion Comparison

The *Palmdale* dataset contains 66 images captured using a fisheye camera lens, such that the images in this sequence contain a strong radial distortion. For comparison purposes, reconstructions were obtained where the radial distortion is present and where the images have been corrected using distortion correction from bundle adjustment results. A trackline visualization was also computed for both reconstructions. The results of the experiment are presented in Figure 5. It can be observed for the distorted reconstruction that feature tracks degrade very quickly over time and the reconstruction has higher reprojection error. For the undistorted case, fewer feature tracks are present due to the difficultly in matching the undistorted images, which get 'stretched' and blurred towards the edges. However, the reconstruction has a lower total reprojection error and the tracks do not degrade as quickly as in the distorted case. Furthermore, notice how the reconstruction using distorted images results in a curve of the ground plane of the scene. After distortion correction, the plane of the scene looks much flatter, as it is in reality.

E. Feature Tracking Algorithm Comparison

1) SIFT vs. SURF: The SIFT [2] and SURF [3] feature tracking algorithms were analyzed and compared on two different datasets: *Palmdale* and *Medusa* [23]. In general, SURF produced less matches than SIFT and reconstructions using SURF degraded very quickly. SIFT tracks are generally more accurate as SIFT features use a 128-dimensional vector to describe each feature as opposed to SURF's 64-dimensional vector, which produces faster results at the cost of accuracy. With SIFT and the *Medusa* dataset, as shown in Figure 6, 1500-2000 matches between pairwise images were obtained, but only 500-800 matches with SURF. In the *Palmdale* dataset, SIFT acquired between 2000-4000 matches per image pair, but SURF only generated about 500-1000.

2) BRISK vs. ORB vs. SIFT vs. SURF: For this test, four different feature tracking algorithms (BRISK [24], ORB [25], SIFT [2], SURF [3]) were run on the *Temple* dataset [26], which consists of six images of a plaster reconstruction of a temple ruin. The feature tracking algorithms were implemented in the OpenCV [27] library. Figure 7 displays the sparse trackline summary visualization plotting Recker et. al's [17] error metric along the tracklines. Both SIFT and SURF produced more accurate tracks than BRISK and ORB, however in all cases drift in the tracking algorithms causes increased angular error.

V. Conclusions

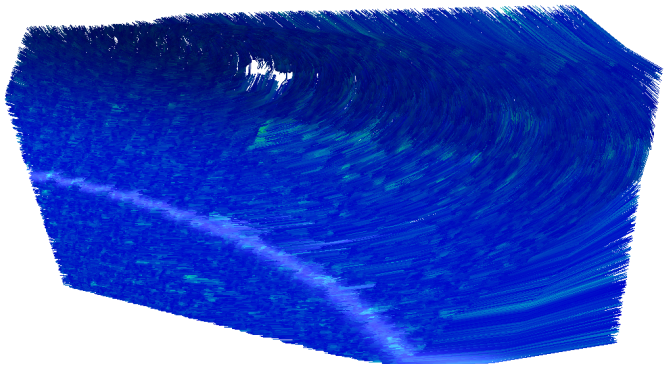
The 'tracklines' visualization technique was presented, which allows users to analyze errors in multi-view sequential scene reconstruction. By computing feature track summaries, researchers can easily observe the progression of feature tracks through a set of frames over time. These summaries easily isolate poor feature tracks and allow the observer to infer the cause of a delinquent track. This visualization technique allows computer vision researchers to analyze errors in ways previously unachieved, and permits insight into different reconstruction phenomena, such as observing feature tracking drift, identifying incorrect tracks, studying the impact of bundle adjustment on a given reconstruction, and analyzing how radial distortion impacts scene reconstruction.

VI. Future Work

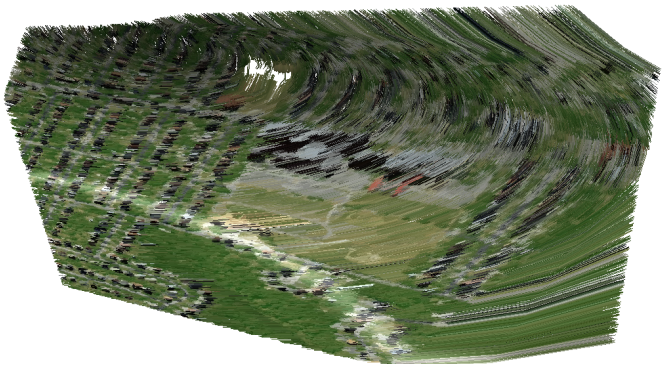
The authors believe continued inter-disciplinary study between visualization and computer vision, such as tracklines, is extremely beneficial for computer vision and can help further advance both fields. It provides interesting data to visualization scientists and improves understanding of vision algorithms' behaviors. With the introduction of visualization to computer vision, additional metrics and visual results can be incorporated into traditionally mathematical analyses. In addition, development in this research would provide valuable educational tools to computer vision instructors. These tools would enhance student understanding of advanced concepts. Below are some specific applications that are based upon the tracklines framework.

A. Trackline Classification

One very important extension to the current framework would be the ability to classify tracklines, such as identifying



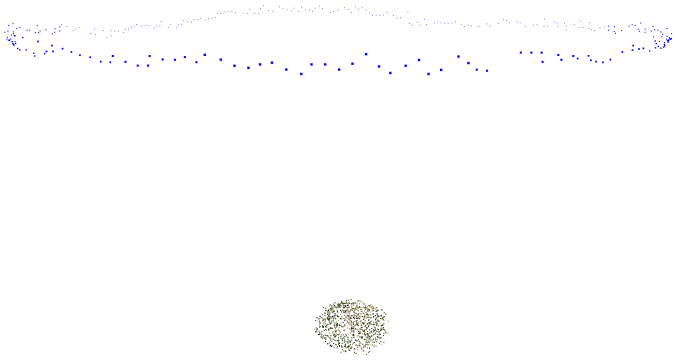
(a) Trackline visualization with reprojection error color map



(b) Trackline visualization with feature track color map



(c) *Megascene1* image used as the first frame in reconstruction



(d) *Megascene1* reconstruction generated with VisualSFM [22]

Fig. 3. Feature track summary visualization of the *Megascene1* dataset (a) using dense correspondences. It is easily observable that higher reprojection errors (shown in green) are present in areas of building occlusions as opposed to the lower blue errors in other regions (b). A sample input image is shown in (c) along with a dense reconstruction in (d) with the camera positions shown in blue.

moving objects, isolating bad tracks, and reporting specular highlights. This can be achieved by the joint analysis of feature track and reprojection error trackline volumes. If a track has a low reprojection error but is inaccurate, then the corresponding 3D structure is necessarily inaccurate. In constrast, if a track is accurate but the intensities show a lot of variation, this is probably due to a moving object or a specular highlight. To distinguish between the two cases, it is more likely a highlight if neighboring tracks show the same behavior. Intensity gradients may provide further clues, particularly if there is any correlation between low gradients and inaccurate tracks. Also, parallax prediction and removal may be necessary, in order to determine which tracks are actually incorrect.

The end result is a set of images with classifications of pixels into movers, correct, incorrect, highlights, and possibly other classifications. It must be kept in mind that this would mainly work well for sparse feature tracking, where tracks are generally unoccluded, which is an important variable that can complicate such an analysis. Furthermore, it may be possible to distinguish between different objects since these should have different 'flows' in trackline-space.

B. Other Applications

There are several other types of analysis that can be achieved using the presented framework. For example, the authors plan on analyzing the relationship between tracklines and Kalman filtering to make predictions of where future feature track positions should lie given the current observations. Applying Principal Component Analysis (PCA) may also be helpful towards characterizing feature track trajectories. With such tools, it may become possible to correct inaccurate tracks such that they follow the correct paths given by accurate tracks. This would allow for the interpolation of virtual views, which is an important step in video stabilization.

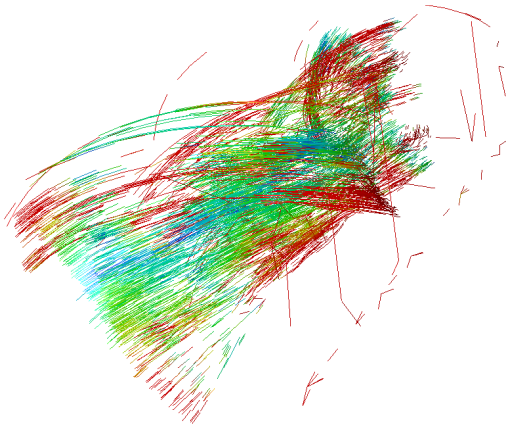
Continuing in this direction of research, feature tracking and reconstruction information could be compressed based on the information provided by the tracklines framework. Dense feature tracking may be possible by using corrected and classified neighboring sparse tracks. It may be possible to create video summaries by representing long tracklines as continuous splines. These are just some examples of the possibilities that can be aided by the trackline representation of scene reconstruction.

ACKNOWLEDGMENT

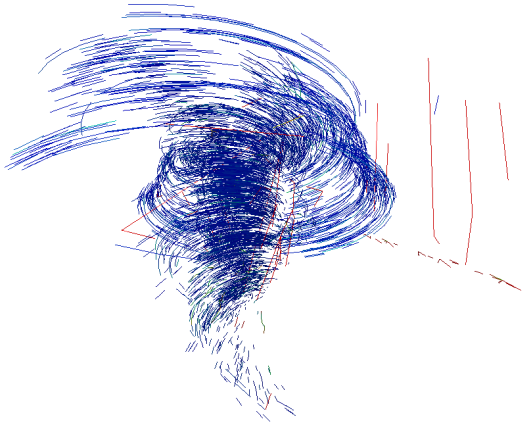
This work was supported in part by Lawrence Livermore National Laboratory and the National Nuclear Security Agency



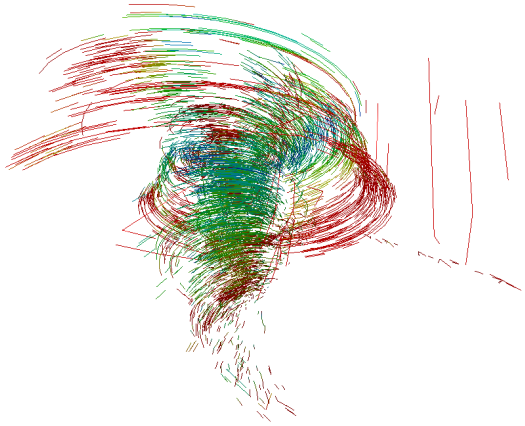
(a) Trackline visualization of a SIFT-based reconstruction of the *Dinosaur* dataset [20] after bundle adjustment (angled view)



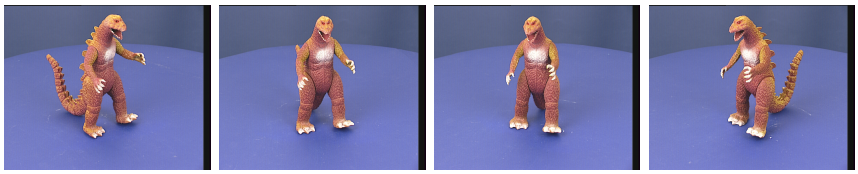
(b) Trackline visualization of a SIFT-based reconstruction of the *Dinosaur* dataset [20] prior to applying bundle adjustment (angled view)



(c) Trackline visualization of a SIFT-based reconstruction of the *Dinosaur* dataset [20] after bundle adjustment (front view)



(d) Trackline visualization of a SIFT-based reconstruction of the *Dinosaur* dataset [20] prior to applying bundle adjustment (front view)



(e) Subset of the *Dinosaur* dataset [20]



(f) *Dinosaur* dataset [20] reconstruction

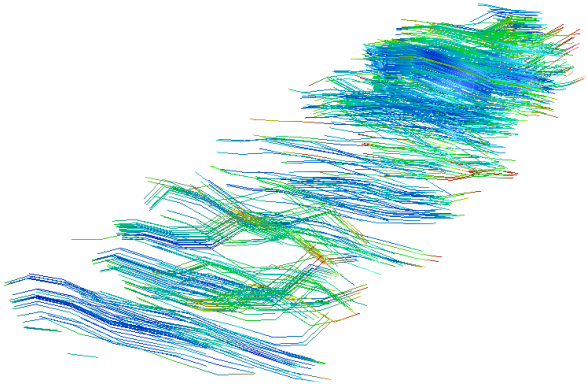
Fig. 4. This figure contains the feature track flow line visualization for the *Dinosaur* dataset [20] reconstruction obtained using SIFT [2] feature tracks, before (*a* and *c*) and after (*b* and *d*) bundle adjustment. It is easily observed that bundle adjustment decreases the reprojection error, where blue values represent low error and red represent high error. Spurious feature tracks are also easily noticed since these tracklines do not follow the movement or flow of the majority of the tracklines. Sample images are displayed in (e) along with a reconstruction in (f).



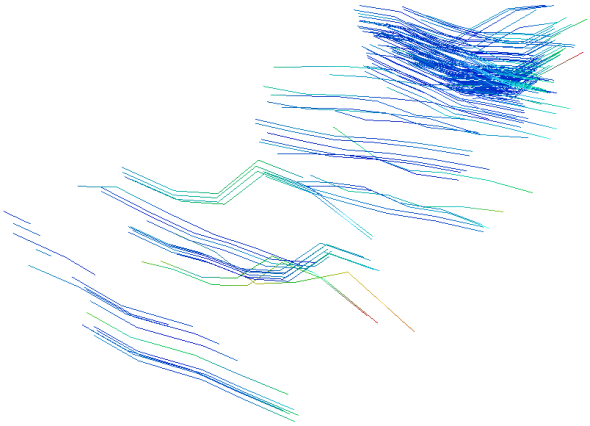
(a) Reconstruction of *Palmdale* with original (distorted) images



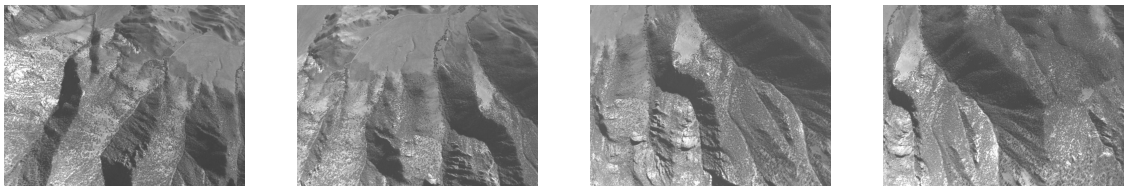
(b) Reconstruction of *Palmdale* with corrected (undistorted) images



(c) Trackline visualization for the reconstruction of *Palmdale* with original (distorted) images

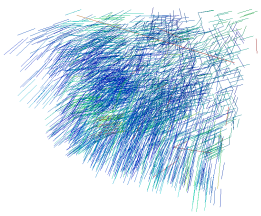


(d) Trackline visualization for the reconstruction of *Palmdale* with corrected (undistorted) images

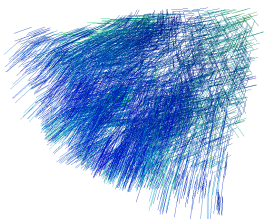


(e) Subset of the *Palmdale* dataset

Fig. 5. This figures contains the reconstructions and trackline visualizations of the *Palmdale* dataset respectively in the presence of radial distortion (*a* and *c*) and when the images have been corrected (*b* and *d*). Sample images are displayed in (e).



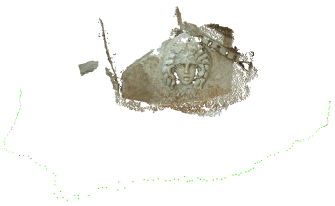
(a) Trackline visualization of a *Medusa* dataset [23] reconstruction using SURF [3]



(b) Trackline visualization of a *Medusa* dataset [23] reconstruction using SIFT [2]

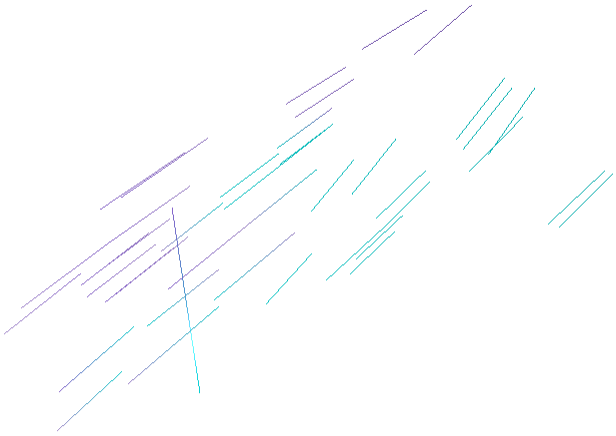


(c) First frame of the *Medusa* dataset [23]

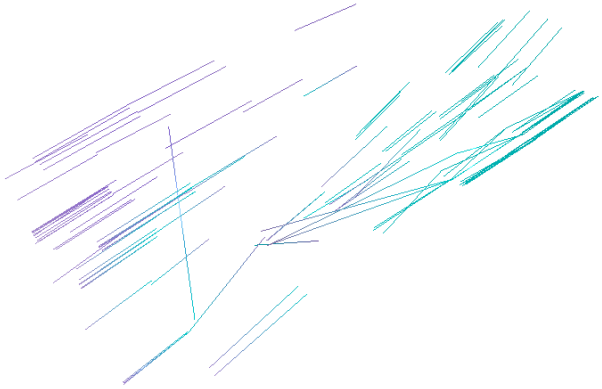


(d) Reconstruction of the *Medusa* dataset [23]

Fig. 6. This figure contains the trackline visualization for the *Medusa* dataset [23] reconstruction obtained with both SURF [3] (a) and SIFT [2] (b) feature tracks. Clearly, the SIFT reconstruction contains more feature matches and is more accurate when compared to SURF. Interestingly, both images showcase feature tracking drift or degradation over time present in these algorithms; as evident by the 'flow' from low error (in blue) to higher error (in green). A sample image from this sequence is shown in (c), with a reconstruction in (d).



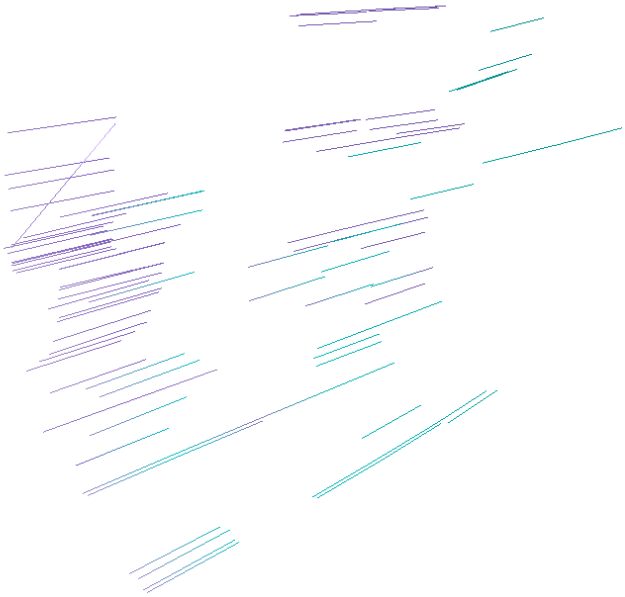
(a) Trackline visualization of a reconstruction of the *Temple* dataset [26] using BRISK [24]



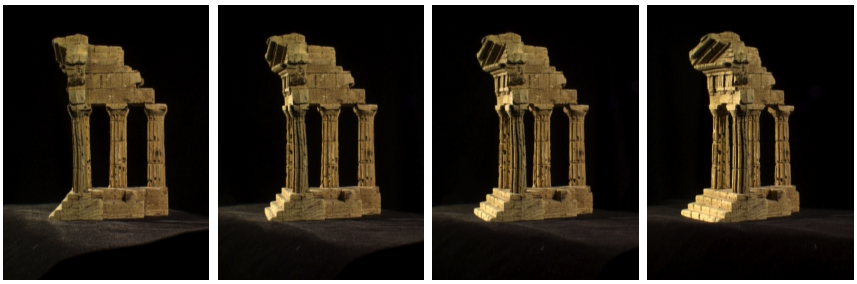
(b) Trackline visualization of a reconstruction of the *Temple* dataset [26] using ORB [25]



(c) Trackline visualization of a reconstruction of the *Temple* dataset [26] using SIFT [2]



(d) Trackline visualization of a reconstruction of the *Temple* datase [26]t using SURF [3]



(e) Subset of the *Temple* dataset [26]



(f) Reconstruction of the *Temple* dataset [26]

Fig. 7. BRISK [24] (a), ORB [25] (b), SIFT [2] (c), and SURF [3] (d) were executed on the *Temple* dataset and trackline visualizations were created using Recker et. al's error metric [17]. Purple and blue represent areas of low angular error and bright green areas have a higher angular error. Sample images are displayed in (e) and a reconstruction in (f).

through Contract No. DE-FG52-09NA29355. The authors thank their colleagues in the Institute for Data Analysis and Visualization (IDAV) at UC Davis for their support.

REFERENCES

[1] R. I. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd ed., 2004.

[2] D. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal On Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[3] H. Bay, T. Tuytelaars, and L. Van Gool, "Surf: Speeded up robust features," in *Computer Vision – ECCV 2006* (A. Leonardis, H. Bischof, and A. Pinz, eds.), vol. 3951 of *Lecture Notes in Computer Science*, pp. 404–417, Springer Berlin / Heidelberg, 2006. 10.1007/11744023.32.

[4] S. S. Beauchemin and J. L. Barron, "The computation of optical flow," *ACM Comput. Surv.*, vol. 27, pp. 433–466, September 1995.

[5] B. Triggs, P. McLauchlan, R. I. Hartley, and A. Fitzgibbon, "Bundle adjustment - a modern synthesis," in *ICCV '99: Proceedings of the International Workshop on Vision Algorithms*, (London, UK), pp. 298–372, Springer-Verlag, 2000.

[6] M. Lourakis and A. Argyros, "The design and implementation of a generic sparse bundle adjustment software package based on the Levenberg-Marquardt algorithm," Tech. Rep. 340, Institute of Computer Science - FORTH, Heraklion, Crete, Greece, August 2000.

[7] A. Akbarzadeh, J.-M. Frahm, P. Mordohai, B. Clipp, C. Engels, D. Gallup, P. Merrell, M. Phelps, S. Sinha, B. Talton, L. Wang, Q. Yang, H. Stewenius, R. Yang, G. Welch, H. Towles, D. Nister, and M. Pollefeys, "Towards urban 3d reconstruction from video," in *3D Data Processing, Visualization, and Transmission, Third International Symposium on*, pp. 1 –8, june 2006.

[8] M. Pollefeys, D. Nistér, J.-M. Frahm, A. Akbarzadeh, P. Mordohai, B. Clipp, C. Engels, D. Gallup, S.-J. Kim, P. Merrell, C. Salmi, S. Sinha, B. Talton, L. Wang, Q. Yang, H. Stewénius, R. Yang, G. Welch, and H. Towles, "Detailed real-time urban 3d reconstruction from video," *International Journal of Computer Vision*, vol. 78, pp. 143–167, 2008. 10.1007/s11263-007-0086-4.

[9] M. Goesele, N. Snavely, C. Curless, H. Hoppe, and S. M. Seitz, "Multi-view stereo for community photo collections," in *Proceedings of ICCV 2007*, 2007.

[10] M. J. Brooks, W. Chojnacki, D. Gawley, and A. van den Hengel, "What value covariance information in estimating vision parameters?," in *ICCV'01*, pp. 302–308, 2001.

[11] L. Cheong, C. Fermüller, and Y. Aloimonos, "Effects of Errors in the Viewing Geometry on Shape Estimation," *Comput. Vis. Image Underst.*, vol. 71, no. 3, pp. 356–372, 1998.

[12] W. Zhao and N. Nandhakumar, "Effects of Camera Alignment Errors on Stereoscopic Depth Estimates," *Pattern Recognition*, vol. 29, pp. 2115–2126, December 1996.

[13] V. Rodehorst, M. Heinrichs, and O. Hellwich, "Evaluation of relative pose estimation methods for multi-camera setups," in *International Archives of Photogrammetry and Remote Sensing (ISPRS '08)*, (Beijing, China), pp. 135–140, 2008.

[14] D. Knoblauch, M. Hess-Flores, M. A. Duchaineau, K. I. Joy, and F. Kuester, "Non-parametric sequential frame decimation for scene reconstruction in low-memory streaming environments," in *ISVC 2011* (M. Hess-Flores, ed.), vol. 6938 of *LNCS*, pp. 363–374, 2011.

[15] T. Torsney-Weir, A. Saad, T. Möller, H.-C. Hege, B. Weber, and J.-M. Verbavatz, "Tuner: Principled parameter finding for image segmentation algorithms using visual response surface exploration," *IEEE Trans. on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 1892–1901, 2011.

[16] A. Saad, T. Möller, and G. Hamarneh, "Probexplorer: Uncertainty-guided exploration and editing of probabilistic medical image segmentation," *Computer Graphics Forum*, vol. 29, no. 3, pp. 1113–1122, 2010.

[17] S. Recker, M. Hess-Flores, M. A. Duchaineau, and K. I. Joy, "Visualization of scene structure uncertainty in a multi-view reconstruction pipeline," in *Vision, Modeling and Visualization Workshop*, pp. 183–190, 2012.

[18] C. Beder and R. Steffen, "Determining an initial image pair for fixing the scale of a 3d reconstruction from an image sequence," in *Proceedings of the 28th conference on Pattern Recognition*, DAGM'06, (Berlin, Heidelberg), pp. 657–666, Springer-Verlag, 2006.

[19] O. Mallo, R. Peikert, C. Sigg, and F. Sadlo, "Illuminated Lines Revisited," in *IEEE Visualization*, pp. 19–26, 2005.

[20] Oxford Visual Geometry Group, "Multi-view and Oxford Colleges building reconstruction," August 2009.

[21] M. Duchaineau, J. Cohen, and S. Vaidya, "Toward fast computation of dense image correspondence on the gpu," in *High Performance Embedded Computing, 2007. HPEC 2007.*, pp. 91–92, 2007.

[22] Changchang Wu, "VisualSfM: A visual structure from motion system." http://homes.cs.washington.edu/ ccwu/vsfm/, 2011.

[23] M. Pollefeys, L. Van Gool, M. Vergauwen, F. Verbiest, K. Cornelis, J. Tops, and R. Koch, "Visual modeling with a hand-held camera," *International Journal of Computer Vision*, vol. 59, pp. 207–232, 2004. 10.1023/B:VISI.0000025798.50602.3a.

[24] S. Leutenegger, M. Chli, and R. Siegwart, "Brisk: Binary robust invariant scalable keypoints," in *Computer Vision (ICCV), 2011 IEEE International Conference on*, pp. 2548–2555, 2011.

[25] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "Orb: An efficient alternative to sift or surf," in *Computer Vision (ICCV), 2011 IEEE International Conference on*, pp. 2564–2571, 2011.

[26] S. Seitz, B. Curless, J. Diebel, D. Scharstein, and R. Szeliski, "A comparison and evaluation of multi-view stereo reconstruction algorithms," in *Computer Vision and Pattern Recognition, 2006 IEEE Computer Society Conference on*, vol. 1, pp. 519–528, 2006.

[27] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.